

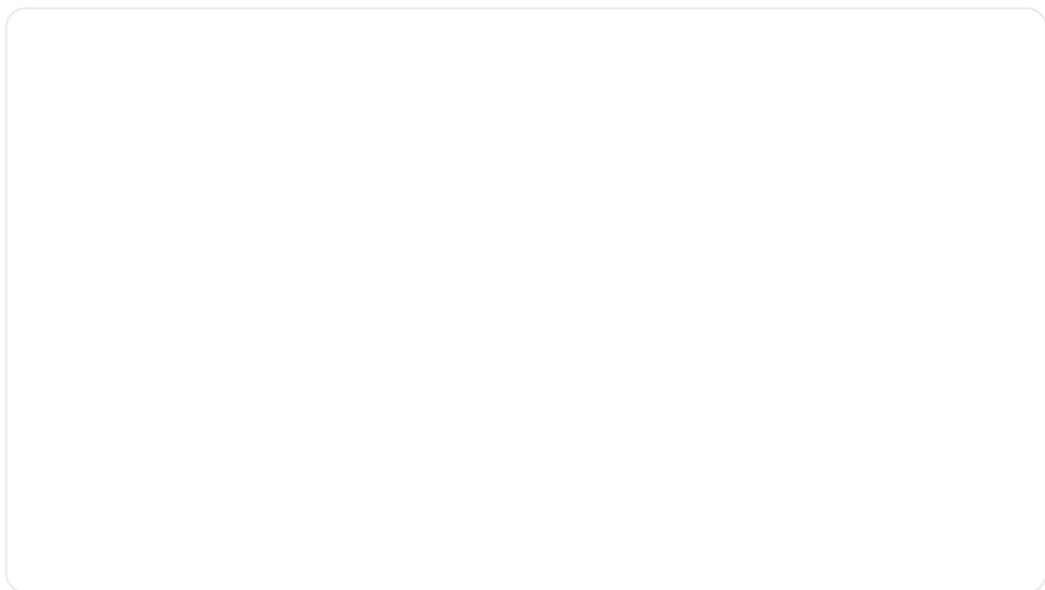


YT to Notes AI

COMPLETE STUDY BUNDLE



Key Topics: Graph Representation



Format: Detailed Notes, Summary, Flashcards, Mind Map Video ID: bxbrUGvBCnY

Quick Summary & Key Takeaways

TL;DR: Graph Representation

- Graphs can be represented using Adjacency Matrix (2D array) or Adjacency List (array of lists/vectors).
- Adjacency Matrix offers $O(1)$ time for adding, removing, and checking edges but uses $O(V^2)$ space.
- Adjacency List uses $O(V + E)$ space, which is more efficient for sparse graphs, but operations like removing or checking edges can be $O(V)$ in the worst case.
- Adjacency List is generally preferred for sparse graphs (like social networks) due to significant space savings.
- Trees are a specific type of graph with $N-1$ edges, no cycles, and a hierarchical structure, whereas general graphs are more flexible.
- Directed graphs use In-degree (incoming edges) and Out-degree (outgoing edges) to describe node connectivity, useful for applications like spam detection.

Understanding these representation methods is fundamental for efficiently working with graph algorithms and choosing the right structure based on graph density and required operations.

Lecture Outline & Core Concepts

📌 Key Topics: Graph Representation

This lecture focuses on methods to represent graphs in a computer system using various data structures. Understanding graph representation is crucial for efficient graph algorithm implementation.

1. Introduction to Graph Representation

- **Purpose:** To store graph information (vertices and edges) in a system using appropriate data structures.
- **Analogy:** Similar to how an array `int arr[5]` represents an array in C++.
- **Key Information Provided for a Graph:**
 - **Vertices (Nodes):** Number of vertices (e.g., 0, 1, 2, 3, 4 for 5 vertices).
 - **Edges:** Connections between vertices (e.g., 0-1, 1-2). For undirected graphs, 0-1 is the same as 1-0.

2. Adjacency Matrix

Definition: A 2D array (matrix) where `matrix[i][j]` indicates if an edge exists between vertex `i` and vertex `j`.

2.1. Unweighted, Undirected Graph

- **Structure:** A $V \times V$ matrix (where V is the number of vertices).
- **Representation:**
 - `matrix[i][j] = 1` if an edge exists between `i` and `j`.
 - `matrix[i][j] = 0` if no edge exists.
- **Symmetry:** For undirected graphs, `matrix[i][j]` will be equal to `matrix[j][i]` (symmetric matrix).
- **Self-loops:** `matrix[i][i]` is typically 0 (no edge from a node to itself, unless specified).
- **Example:** For vertices 0-4, if 0-1 has an edge, then `matrix[0][1] = 1` and `matrix[1][0] = 1`.

2.2. Weighted, Undirected Graph

- **Structure:** Same $V \times V$ matrix.
- **Representation:**
 - `matrix[i][j] = weight` if an edge exists between `i` and `j` with a specific weight.
 - `matrix[i][j] = 0` if no edge exists (or a special value like infinity if 0 could be a valid weight).
- **Symmetry:** `matrix[i][j] = weight` implies `matrix[j][i] = weight`.

Symmetry: $\text{matrix}[i][j] = \text{matrix}[j][i]$ if no directed edge exists.

2.3. Unweighted, Directed Graph

- **Structure:** Same $V \times V$ matrix.
- **Representation:**
 - $\text{matrix}[i][j] = 1$ if a directed edge exists *from* i *to* j .
 - $\text{matrix}[i][j] = 0$ if no directed edge exists.
- **Asymmetry:** $\text{matrix}[i][j]$ is generally *not* equal to $\text{matrix}[j][i]$. An edge from i to j does not imply an edge from j to i .

2.4. Weighted, Directed Graph

- **Structure:** Same $V \times V$ matrix.
- **Representation:**
 - $\text{matrix}[i][j] = \text{weight}$ if a directed edge exists *from* i *to* j with a specific weight.
 - $\text{matrix}[i][j] = 0$ if no directed edge exists.

2.5. Time and Space Complexity (Adjacency Matrix)

- **Space Complexity:** $O(V^2)$ because it requires a $V \times V$ matrix to store information for V vertices.
- **Time Complexity (Creation):** $O(V^2)$ to initialize the matrix.
- **Time Complexity (Add Edge):** $O(1)$ to set $\text{matrix}[u][v] = 1$ (and $\text{matrix}[v][u] = 1$ for undirected).
- **Time Complexity (Remove Edge):** $O(1)$ to set $\text{matrix}[u][v] = 0$ (and $\text{matrix}[v][u] = 0$ for undirected).
- **Time Complexity (Check Edge Existence):** $O(1)$ to check $\text{matrix}[u][v]$.

3. Adjacency List

Definition: An array of lists (or vectors in C++) where the i -th element of the array contains a list of all vertices adjacent to vertex i .

3.1. Unweighted, Undirected Graph

- **Structure:** `vector<int> adj[V]` (an array of vectors, each vector stores integers).
- **Representation:** For an edge $u-v$:
 - Add v to $\text{adj}[u]$'s list.
 - Add u to $\text{adj}[v]$'s list (for undirected graphs).
- **Example:** If 0-1 and 0-2 have edges, $\text{adj}[0]$ would contain $[1, 2]$. $\text{adj}[1]$ would contain $[0]$, $\text{adj}[2]$ would contain $[0]$ (plus other connections).

3.2. Weighted, Undirected Graph

- **Structure:** `vector<pair<int, int>> adj[V]` (an array of vectors, each vector stores pairs of (neighbor_vertex, weight)).
- **Representation:** For an edge $u-v$ with weight w :
 - Add (v, w) to $\text{adj}[u]$'s list.
 - Add (u, w) to $\text{adj}[v]$'s list (for undirected graphs).

3.3. Time and Space Complexity (Adjacency List)

- **Space Complexity:** $O(V + E)$ where V is the number of vertices and E is the number of edges. This is because we store V lists, and the total number of elements across all lists is $2E$ for undirected graphs (each edge $u-v$ is stored twice: v in $\text{adj}[u]$ and u in $\text{adj}[v]$).
- **Time Complexity (Creation):** $O(V + E)$ to initialize the lists and add all edges.
- **Time Complexity (Add Edge):** $O(1)$ on average for push_back operation (amortized constant time)

Time Complexity (Add Edge): $O(1)$ on average for `push_back` operation (amortized constant time).

- **Time Complexity (Remove Edge):** $O(\text{degree}(u))$ or $O(V)$ in worst case, as it might require searching through the list of neighbors and shifting elements.
- **Time Complexity (Check Edge Existence):** $O(\text{degree}(u))$ or $O(V)$ in worst case, as it requires iterating through the list of neighbors of u .

4. Comparison: Adjacency Matrix vs. Adjacency List

Feature	Adjacency Matrix	Adjacency List
Add Edge	$O(1)$	$O(1)$ (amortized)
Remove Edge	$O(1)$	$O(\text{degree}(V))$ or $O(V)$
Check Edge	$O(1)$	$O(\text{degree}(V))$ or $O(V)$
Space Complexity	$O(V^2)$	$O(V + E)$

- **Dense Graphs:** Graphs with a large number of edges (E is close to V^2). Adjacency Matrix is often preferred because its $O(V^2)$ space is fully utilized, and $O(1)$ operations are beneficial.
- **Sparse Graphs:** Graphs with a small number of edges (E is much less than V^2). Adjacency List is preferred because its $O(V + E)$ space is significantly less than $O(V^2)$, saving memory. (e.g., Social networks like Facebook: billions of users (V), but each user has a limited number of friends ($E \ll V^2$)).
- **General Preference:** In competitive programming and for most real-world applications, Adjacency List is more commonly used due to its better space efficiency for sparse graphs, which are prevalent.
- **Optimization:** Using `unordered_map` for adjacency lists can further optimize average-case time complexities for operations like `remove edge` and `check edge existence` to $O(1)$.

5. Tree vs. Graph

Definition: A tree is a specific type of graph with particular properties.

Feature	Tree	Graph
Edges	Exactly $N-1$ edges for N nodes	Any number of edges (0 to $N*(N-1)/2$)
Loops/Cycles	No loops/cycles	Can contain loops/cycles
Connectivity	Always connected	Can be connected or disconnected
Root	Has a designated root node	No concept of a root node
Parent-Child	Each child has exactly one parent (except root)	A node can have multiple 'parents' (edges) or none
Hierarchy	Hierarchical structure	Network structure

- **Key Takeaway:** Every tree is a graph, but not every graph is a tree.
- **Example of Graph (not a Tree):** A graph with isolated nodes (no edges) is a valid graph but not a tree because it lacks $N-1$ edges.

6. Degree of a Node

Definition: The number of edges connected to a node.

- **Undirected Graph:** Degree = total number of edges connected to the node.
- **Directed Graph:**
 - **In-degree:** Number of incoming edges to a node.
 - **Out-degree:** Number of outgoing edges from a node.
- **Application:** Used in social networks (e.g., followers vs. following) and for detecting spam accounts (e.g., very high out-degree with low in-degree might indicate a bot).

💡 Important Definitions:

- **Vertex (Node):** A fundamental unit of a graph, representing an entity.
- **Edge:** A connection between two vertices.
- **Undirected Graph:** A graph where edges have no direction (connection is bidirectional).
- **Directed Graph:** A graph where edges have a specific direction (connection is unidirectional).
- **Unweighted Graph:** A graph where edges do not have associated costs or values.
- **Weighted Graph:** A graph where edges have associated costs or values (weights).
- **Adjacency Matrix:** A 2D array representation of a graph.
- **Adjacency List:** An array of lists/vectors representation of a graph.
- **Dense Graph:** A graph with many edges, close to the maximum possible ($E \approx V^2$).
- **Sparse Graph:** A graph with few edges, significantly less than the maximum possible ($E \ll V^2$).
- **Complete Graph:** A graph where every pair of distinct vertices is connected by a unique edge.
- **Tree:** A connected acyclic graph.
- **In-degree:** Number of edges pointing towards a node in a directed graph.
- **Out-degree:** Number of edges pointing away from a node in a directed graph.

Flashcards & Self-Testing

Q: What are the two primary methods for graph representation?

A: The two primary methods are Adjacency Matrix and Adjacency List.

Q: What is the space complexity of an Adjacency Matrix for a graph with V vertices?

A: The space complexity is $O(V^2)$.

Q: What is the space complexity of an Adjacency List for a graph with V vertices and E edges?

A: The space complexity is $O(V + E)$.

Q: When is an Adjacency Matrix generally preferred over an Adjacency List?

A: An Adjacency Matrix is preferred for dense graphs where the number of edges (E) is close to V^2 .

Q: When is an Adjacency List generally preferred over an Adjacency Matrix?

A: An Adjacency List is preferred for sparse graphs where the number of edges (E) is much less than V^2 , due to better space efficiency.

Q: What is the key difference in edge count between a tree and a general graph with N nodes?

A: A tree with N nodes always has exactly $N-1$ edges, while a general graph can have any number of edges from 0 to $N*(N-1)/2$.

Q: Define In-degree and Out-degree in a directed graph.

Q: Define in-degree and Out-degree in a directed graph.

A: In-degree is the number of edges pointing towards a node. Out-degree is the number of edges pointing away from a node.

Q: Can every graph be a tree? Explain.

A: No. While every tree is a graph, not every graph is a tree. Graphs can have cycles, more than $N-1$ edges, or be disconnected, which are not properties of a tree.

Visual Mind Map & Relationships

Graph Representation

Adjacency Matrix

Concept: 2D array (V x V)